

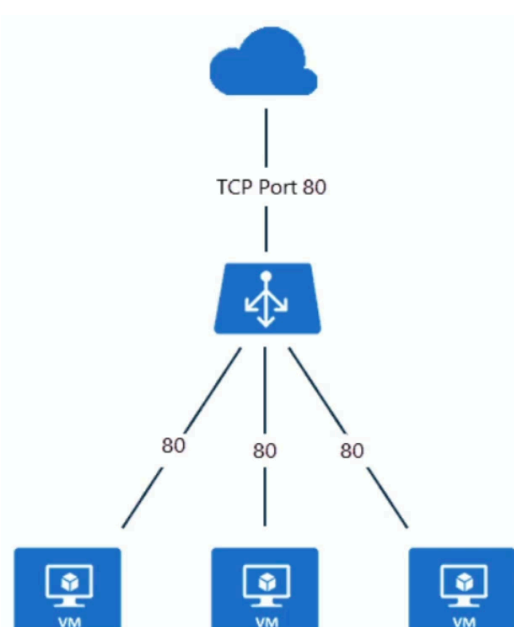
Section 6: Scalability in System Design

Understanding Load Balancers

In this session we will learn how load balancers distribute traffic, improve availability, and form one of the most critical building blocks of scalable and resilient system architecture. So, let's get started.

Why is Load Balancing Needed?

Now as systems grow, the challenge is no longer just serving requests, it is serving them reliably under unpredictable traffic. A single server might work initially, but eventually it becomes both a performance bottleneck and a single point of failure. Load balancing solves this by introducing an intelligent traffic distribution layer between users and application servers. Instead of allowing one machine to handle everything, requests are now spread across multiple servers, improving the resource utilization, and preventing overload. The result is better response times, higher throughput, and a more consistent user experience.



Now, just as importantly, load balancing improves resilience too. In production, servers fail, deployments go wrong, and infrastructure issues occur. A load balancer continuously checks server health and automatically routes traffic away from unhealthy instances, helping the system remain available during failures. It also becomes a key enabler for scalability. As demand increases, we can add more servers behind the load balancer without changing how users access the application. This allows capacity to grow incrementally rather than relying on a single larger machine.

Think about a major e-commerce platform during a flash sale. Millions of requests may arrive within minutes. Without load balancing, a few servers could quickly become overwhelmed. With a load balancer in place, traffic is distributed across a fleet of servers, ensuring that the platform remains fast, available, and capable of handling peak demand. That is why load balancing is one of the foundational building blocks of scalable system design.

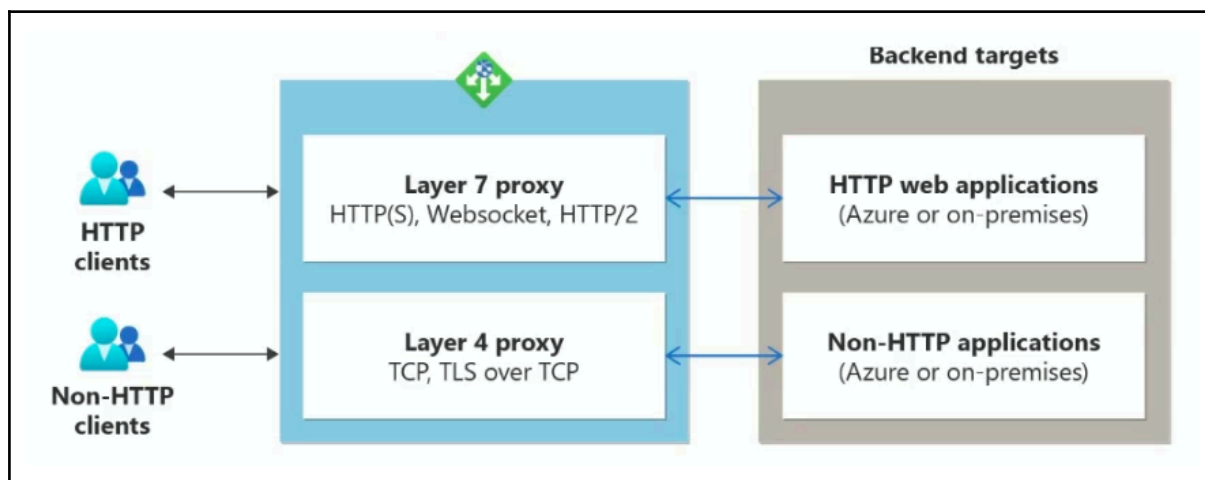
Types of Load Balancers

As system scale, not all load balancers solve the problem in the same way. The right choice depends on two key questions:

- How deeply should load balancers inspect the traffic?
- Where should it be deployed?

Based on Layer:

The first classification is based on the network layer where routing decisions are made. Layer 4 load balancers operate at TCP and UDP level. They only look at the information such as IP addresses and ports for making routing decisions without inspecting the actual request. Because they process less information. They are extremely fast and are often used for high-throughput, low-latency workload. But layer 7 load balancers operate at the application layer and they understand the protocols like HTTP and HTTPS. They can examine URLs, headers, cookies, and other request attributes to make more intelligent routing decisions. For example, a request to slash checkout might be routed to dedicated transaction service, while a request to slash product is sent to a different background server. This flexibility comes with additional processing overhead, but it enables much more smarter traffic management.



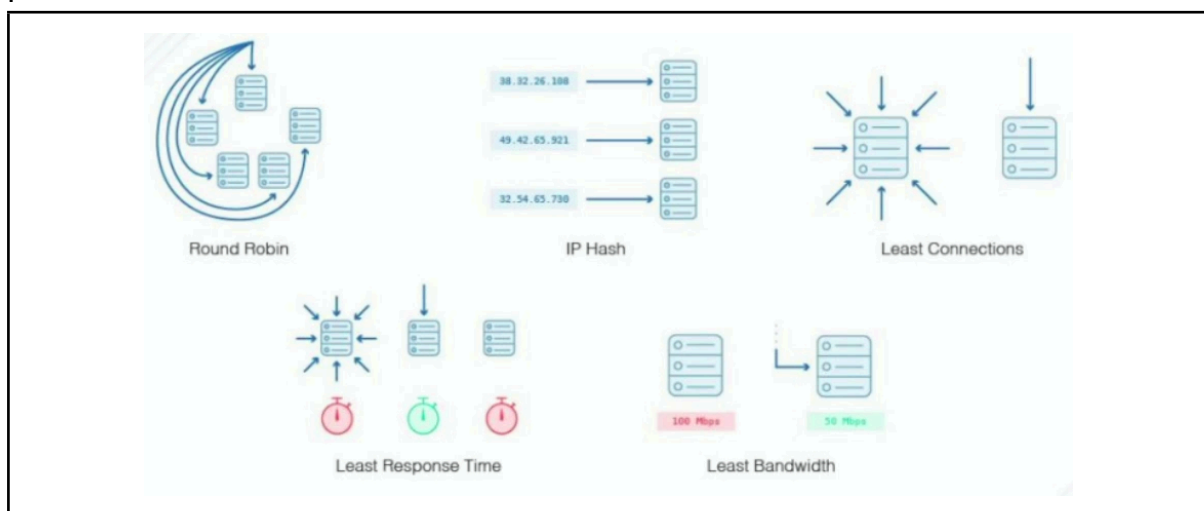
Based on Deployment:

Load balancers can also be categorized by deployment mode. You can have hardware load balancers and software load balancers. Hardware load balancers are dedicated appliances commonly found in larger enterprise data centers, where maximum performance and specialized networking features are required. Software load balancers such as Nginx, HAProxy, and Envoy run on standard servers or containers. Their flexibility, lower cost, and cloud-native compatibility have made them the default choice for many modern architectures. And finally, cloud providers offer fully managed load balancing services too. Solutions such as AWS Elastic Load Balancer, Google Cloud Cloud Balancing, and Azure Load Balancer remove much of the operational burden by automatically handling scaling, health check and traffic distribution.

The key takeaway is that Layer 4 vs Layer 7 determines how traffic is routed, while hardware, software, and cloud-based options determine where and how the load balancer is deployed. As architects, we choose the combination that best balances performance, flexibility, operational complexity, and cost

Load Balancing Strategies

Once traffic reaches multiple servers, the next question is how do we decide which server should handle each request? That decision is governed by a load balancing strategy. Some strategies are rule-based and relatively very simple.

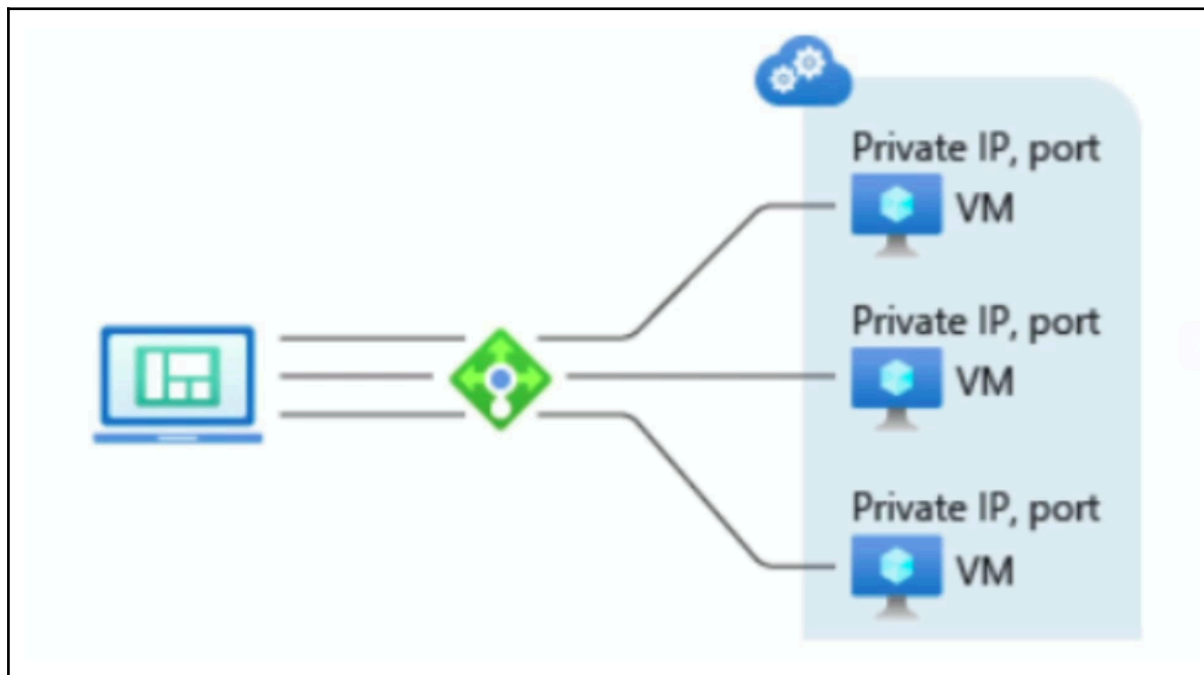


Round Robin distributes requests in a fixed sequence across available servers. It works well when the servers have similar capacity and workloads are predictable. Then least connection takes one step further by considering active connections, making it more suitable when some requests take significantly longer to process. IP hashing introduces consistency by routing the same client to the same backend server, which can be useful for session persistence and sticky sessions. The challenge with these approaches is that they assume that the system's state is relatively stable. In a production environment, that is rarely true. Traffic patterns change, servers experience different workloads, and infrastructure conditions evolve continuously, that is where Dynamic Load Balancing becomes valuable.

Instead of following predefined rules the load balancers make decisions based on real-time conditions. A least response times strategy favors servers currently responding the fastest. Then Adaptive Load Balancing goes even further by considering metrics such as CPU utilization, memory pressure, server health, and traffic pattern. Then weighted load balancing allows more powerful servers to receive a larger share of requests, ensuring that infrastructure capacity is utilized efficiently.

As architects, the goal for us is not to find the best algorithm, but the most appropriate algorithm for the current workload. Simple strategies are easier to operate and often sufficient at a smaller scale, while dynamic strategies provide better resource utilization and resilience as systems become larger and more unpredictable. The more variable your traffic and infrastructure is, the more valuable intelligent load balancing becomes for you.

Load Balancer in Action

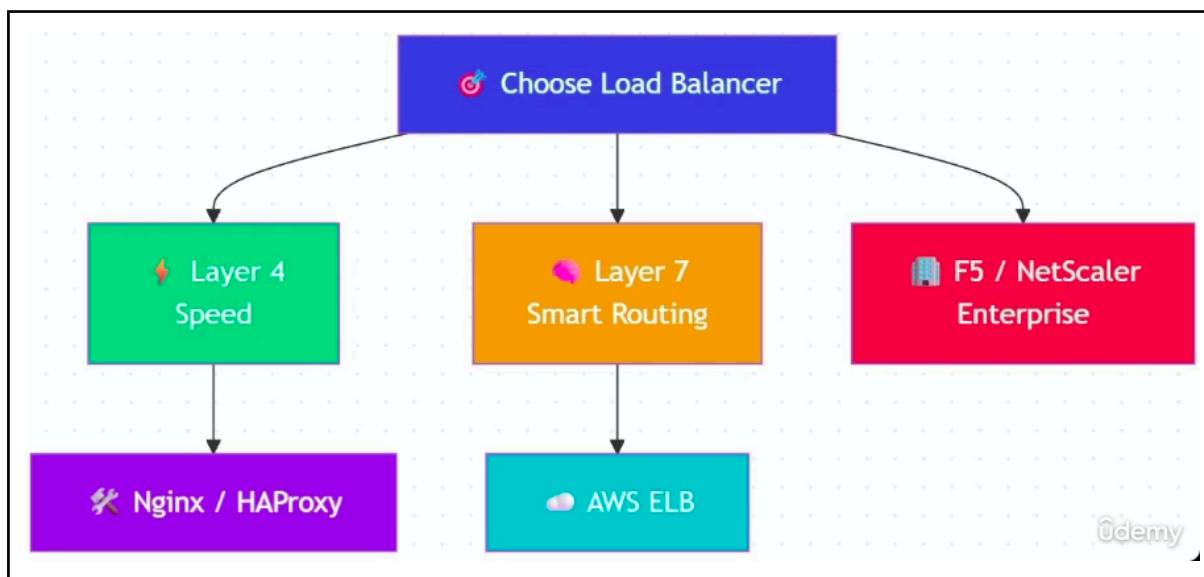


Let's bring everything together now with a simple request flow. A user sends a request to the application's public endpoint. The load balancer receives that request first and becomes the single entry point for the system. Now based on a configured routing algorithm, such as round-robin, least connection, or more advanced dynamic strategy, it selects an appropriate backend server. From the user's perspective, this entire process is invisible. They interact with a single application endpoint, while the load balancer handles traffic distribution, health check, and failover behind the scenes. This abstraction is one of the key reasons that load balancers are so valuable. They decouple clients from individual servers, allowing infrastructure to scale, recover from failures, and evolve without affecting the users.

Choosing the Right Load Balancer

The first consideration is how much intelligence you need in traffic routing. If your goal is simply to distribute TCP or UDP traffic as efficiently as possible, a layer 4 load balancer is often the right choice. But if your routing decisions depend on URLs, headers, cookies, or application-specific rules, you will typically need a layer 7 load balancer.

Next, think about scale and operational complexity. For many applications, software load balancers such as Nginx or HAProxy provide an excellent balance of flexibility and performance, and cost. As systems move toward cloud-native architecture, managed load balancing services become attractive because they automatically handle scaling, health check, and infrastructure management.



Then security is another important factor. In many architectures, the load balancer becomes the first line of defense. It often performs SSL termination, centralizes the certificate management, and integrates with DDoS protection, web application firewalls, and traffic filtering services too. A useful rule of thumb is to start with the simplest solution that meets your current requirement. Use software load balancers when you need flexibility and control. Cloud managed load balancers when you want operational simplicity and elastic scaling and specialized hardware appliances only when performance, compliance, or networking requirements justify the additional cost and complexity.

The key takeaway is that choosing a load balancer is rarely a networking decision alone. It is a trade-off involving performance, scalability, security, operational overhead, and long-term system evolution.

Summary & Key Takeaways

- Load balancing enhances scalability, reliability, and performance.
- Different types exist based on layers and deployment models.
- Choosing the right strategy depends on traffic patterns and system needs.
- Essential for highly available and resilient architectures.
- What's next:
 - Autoscaling & Best Practices in Cloud Environments

Interview Questions on Load Balancing

- Fundamentals
 - What is load balancing, and why is it important?
 - Explain the difference between Layer 4 and Layer 7 load balancing.
 - How does a load balancer handle high availability and failover?
- Strategies & Use Cases
 - Compare Round Robin and Least Connections strategies.
 - What are the advantages of Weighted Load Balancing?
 - When would you use a software load balancer over a hardware one?
- Real-World Scenarios
 - How would you design a scalable load balancing solution for a large e-commerce site?
 - What factors should be considered when choosing a load balancing strategy?
 - How does a load balancer improve security?